



Cookify

Para quien cocina en casa con un presupuesto contado, Cookify arma **una semana de almuerzos que cabe en lo que puede gastar**, con el precio real del supermercado donde compra.

\$307.947

gasta al mes en comida
el hogar chileno promedio

237,8%

de diferencia por la misma
canasta según el supermercado

69

recetas chilenas
en el catálogo

DOCUMENTO DE PROPUESTA · HITO 1

Proyecto semestral · Arquitectura de Desarrollo (Móvil y Web)

Índice

Introducción	3
1 • El problema y a quién le duele	4
1.1 La comida es el mayor gasto del hogar chileno	4
1.2 El mismo carro cuesta hasta 238 % más según dónde se compre	4
1.3 Quién es y qué hace hoy sin nosotros	5
2 • La propuesta	6
2.1 La promesa	6
2.2 El producto	7
3 • Arquitectura de información	8
4 • Arquitectura técnica	9
4.1 Cliente, servidor y capas	9
4.2 Qué hace cada capa al armar una semana	10
4.3 Por qué el precio necesita un servidor	12
5 • Decisiones de tecnología	14
5.1 Plataforma del cliente	14
5.2 Navegación y renderizado	14
5.3 Persistencia y servidor	15
6 • Lo que viene	18
Conclusión	19
Bibliografía	20
Anexos	21

INTRODUCCIÓN

Todos los domingos alguien hace la misma cuenta mal

Abrir el refrigerador, decidir qué se va a comer esta semana y calcular si la plata alcanza. Se hace de memoria, se hace mal, y se paga la diferencia en el supermercado.

Planificar una semana de comida es un problema aritmético disfrazado de problema doméstico. Hay que elegir siete platos que no se repitan, que le gusten a quienes van a comerlos, que se puedan cocinar con lo que hay en la cocina, y que sumados no pasen de cierta cifra. Nadie tiene ese cálculo en la cabeza, así que en la práctica se resuelve de dos formas: comprando de más por si acaso, o improvisando cada día y comprando cuatro veces por semana.

Las dos salen caras. La primera termina en comida que se bota; la segunda, en compras chicas y frecuentes donde el precio unitario es peor y las decisiones se toman con hambre.

Cookify no propone recetas. Propone una semana completa que cabe en una cifra que el usuario escribió.

Este documento presenta Cookify: qué problema resuelve y con qué evidencia se afirma que existe, a quién está dirigido, cómo se organiza el producto por dentro, qué arquitectura lo sostiene y qué decisiones de tecnología se tomaron, con sus costos asumidos por escrito.

Una aclaración sobre el alcance: lo que queda comprometido para los hitos siguientes está señalado como tal en cada sección donde aparece, para que se pueda distinguir de lo que el documento describe en presente.

CÓMO LEER ESTE DOCUMENTO

Las secciones 1 y 2 son la propuesta comercial: el problema, la evidencia y la promesa. Las secciones 3 a 5 son la respuesta técnica: cómo se organiza la información, qué arquitectura la sostiene y por qué se eligió cada herramienta. La sección 6 dice qué falta y cuándo.

CAPÍTULO 1

El problema y a quién le duele

Tres cifras verificadas en la fuente original delimitan el problema: cuánto pesa, cuánto varía y cuánto se pierde.

1.1 La comida es el mayor gasto del hogar chileno

La IX Encuesta de Presupuestos Familiares del Instituto Nacional de Estadísticas, levantada entre octubre de 2021 y septiembre de 2022 sobre 15.134 hogares de 79 comunas en las 16 regiones del país, midió que el hogar promedio gasta **\$1.451.782** al mes. De esa cifra, **\$307.947 se van en alimentos y bebidas no alcohólicas**: el **21,2 %** del total, y la primera categoría de gasto por delante de vivienda y de transporte.

El dato que importa no es solo el tamaño sino la dirección: esa proporción **creció 2,5 puntos porcentuales** respecto de la encuesta anterior. La comida pesa cada vez más en el presupuesto familiar.

\$307.947

gasto mensual del hogar en
alimentos

21,2 %

del gasto total, la primera
categoría

+2,5 pp

de aumento respecto de la EPF
anterior

Sobre \$307.947 mensuales, equivocarse en un 20 % son **\$61.589 al mes**, o \$739.000 en un año. No es un margen de error tolerable: es una cuota de algo.

1.2 El mismo carro cuesta hasta 238 % más según dónde se compre

El SERNAC publicó el 7 de septiembre de 2026 su Cotizador de Fiestas Patrias, construido sobre **250.000 precios de 1.300 sucursales de 47 locales** —40 supermercados y 7 carnicerías— a lo largo del país. Comparando una canasta de seis productos idénticos:

CANASTA DE SEIS PRODUCTOS IDÉNTICOS	PRECIO
La más barata	\$23.139
El promedio del mercado	\$45.290
La más cara	\$78.197
Diferencia por exactamente los mismos productos	\$55.058 · 237,8 %

Un solo producto de esa canasta, el costillar de cerdo de 1,2 kg, varía **\$14.004** entre su precio mínimo (\$6.588) y su máximo (\$20.592).

LO QUE ESTO SIGNIFICA PARA EL PRODUCTO

El supermercado donde se compra no es un detalle de configuración: es la diferencia entre que el presupuesto alcance o no. Por eso en Cookify es la *primera* pregunta del recorrido y no un ajuste escondido.

1.3 Quién es y qué hace hoy sin nosotros

El usuario objetivo es **quien cocina para su casa y tiene el presupuesto contado**. No es alguien que busca inspiración gastronómica: es alguien que ya sabe que tiene que cocinar y necesita que la cuenta cierre.

Concretamente: un hogar de dos a cinco personas, que hace una compra grande a la semana o cada quincena, que cocina el almuerzo en casa al menos de lunes a viernes, y para quien pasarse del presupuesto tiene consecuencias reales en el resto del mes.

La competencia real no es otra aplicación

Es importante nombrarla bien, porque define contra qué se compara el producto. Hoy esta persona resuelve el problema de tres maneras, y ninguna involucra software:

- **De memoria.** Repite las mismas seis o siete preparaciones que sabe hacer, sin calcular el costo. Funciona, pero aburre y no controla el gasto.
- **Con una lista escrita a mano.** Anota lo que cree que necesita, y descubre el total en la caja. El presupuesto se verifica cuando ya es tarde.
- **Improvisando.** Decide el mismo día y compra a diario. Es lo más caro de todo y lo que más comida bota.

Las aplicaciones de recetas que existen no compiten con esto porque resuelven otro problema: proponen platos, no presupuestos. Ninguna responde la pregunta que esta persona realmente tiene, que no es «¿qué cocino?» sino «¿me alcanza?».

El desperdicio, la otra cara del mismo problema

Chile genera **1,62 millones de toneladas** de residuos de alimentos al año. El estudio *Cuánto alimento desechan los chilenos* de la Universidad de Talca determinó que **el 95 % de las personas considera normal botar la comida acumulada en el refrigerador**.

Ese desperdicio no es principalmente de sobras del plato: es de comida que se compró sin un plan y nunca se cocinó. Es exactamente el desperdicio que ataca comprar para los días y las personas que efectivamente se van a cocinar, que es lo que Cookify calcula.

EL PROBLEMA EN UNA FRASE

Quien cocina en casa con el presupuesto contado no tiene forma de saber, antes de ir al supermercado, si lo que piensa cocinar esta semana cabe en la plata que tiene.

CAPÍTULO 2

La propuesta

2.1 La promesa

Para quien cocina en casa con un presupuesto contado, Cookify arma una semana de almuerzos que cabe en lo que puede gastar, con el precio del supermercado donde compra.

La promesa es verificable, que es lo que la separa de un eslogan. Si el usuario dice que tiene \$50.000, la aplicación entrega una semana cuyo costo total es menor o igual a \$50.000 o le dice exactamente cuánto le falta y cuál es la semana más barata posible. No hay un tercer resultado.

Qué entrega concretamente

- Un plato de almuerzo para cada día que el usuario eligió cocinar, sin repetir.
- El costo de cada día y el total de la semana, contra el presupuesto declarado.
- Para cada plato: tiempo de preparación, clasificación, calorías, ingredientes ya escalados a la cantidad de personas y agrupados por pasillo del supermercado, e instrucciones numeradas.
- La semana guardada, que sigue ahí al cerrar y volver a abrir la aplicación.

Qué no hace, por decisión

Nombrar los límites es parte de la propuesta. Cookify **solo planifica almuerzos**, no las tres comidas: es la comida que más se cocina en casa y la que concentra el gasto. **No tiene cuentas de usuario** ni red social de recetas. **No cotiza en línea en tiempo real**: estima con precios de referencia por cadena, y el Hito 2 reemplaza esa estimación por mediciones reales.

Las siete preguntas

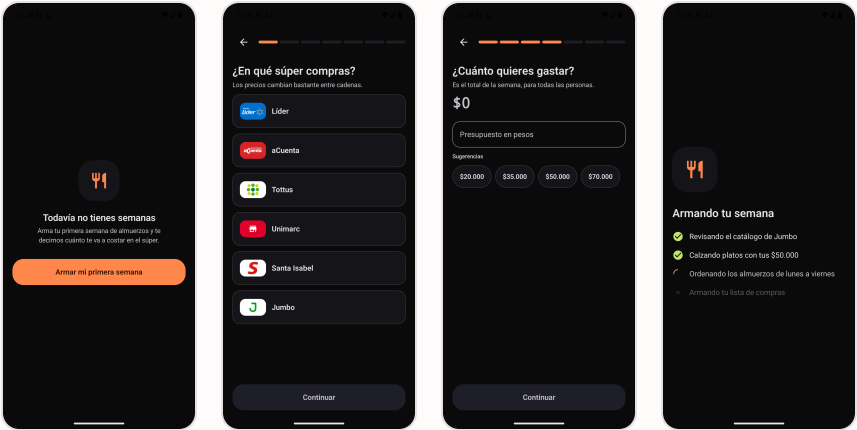
El recorrido de entrada son siete preguntas, una por pantalla, en este orden:

#	PREGUNTA	PARA QUÉ SE USA LA RESPUESTA
1	¿En qué súper compras?	Determina la tabla de precios
2	¿Para cuántas personas?	Escala cada ingrediente y el costo
3	¿Qué días vas a cocinar?	Define cuántos platos y en qué orden
4	¿Cuánto quieres gastar?	Es la restricción que el motor debe cumplir
5	¿Qué se te antoja?	Puntúa y ordena las recetas candidatas
6	¿Alguna restricción?	Filtra el recetario antes de elegir
7	¿Con qué cocinas?	Descarta recetas imposibles de preparar

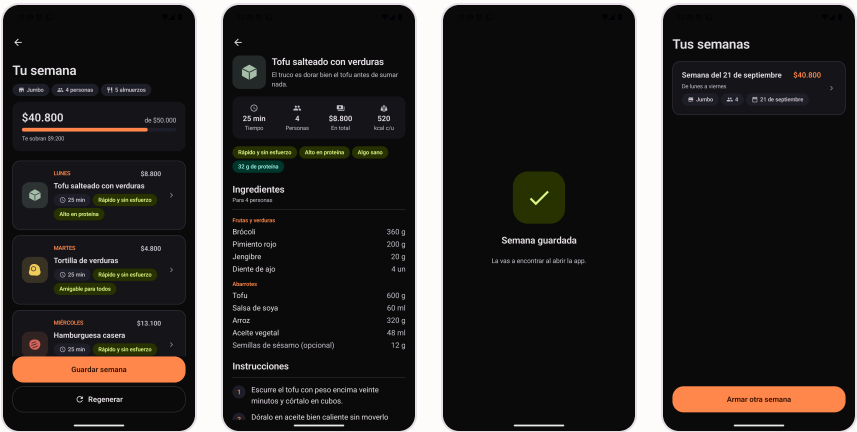
El orden no es casual: las preguntas 6 y 7 *filtran* el recetario y la 4 lo *restringe*, mientras que la 5 solo *ordena*. Preguntar primero lo que más reduce el espacio de búsqueda es lo que permite detectar temprano cuándo una combinación no tiene solución.

2.2 El producto

El recorrido completo, de la portada vacía hasta la semana instalada en ella. Las ocho pantallas salieron de una misma sesión de uso.



Del vacío al armado. La portada sin semanas es una sola invitación. Las siete preguntas se hacen de a una. La espera nombra los datos reales del usuario en vez de mostrar un porcentaje.



El resultado. La semana con su costo contra el presupuesto, el detalle de un plato con los ingredientes escalados y agrupados por pasillo, la confirmación al guardar, y la semana instalada en la portada.

Estado por componente

COMPONENTE	ENTREGA
Recorrido completo, de la portada vacía a la semana guardada	Hito 1
Recetario de 69 platos chilenos y 98 ingredientes con precio	Hito 1
Motor de planificación determinista, probado con JUnit	Hito 1
Persistencia local en Room 3, sobrevive al cierre	Hito 1
Manejo de presupuesto insuficiente y de catálogo insuficiente	Hito 1
Servicio de precios reales por cadena	Hito 2
Lista de compras consolidada de la semana	Hito 2

CAPÍTULO 3

Arquitectura de información

Cómo se agrupa, se jerarquiza y se nombra todo lo que el producto contiene.

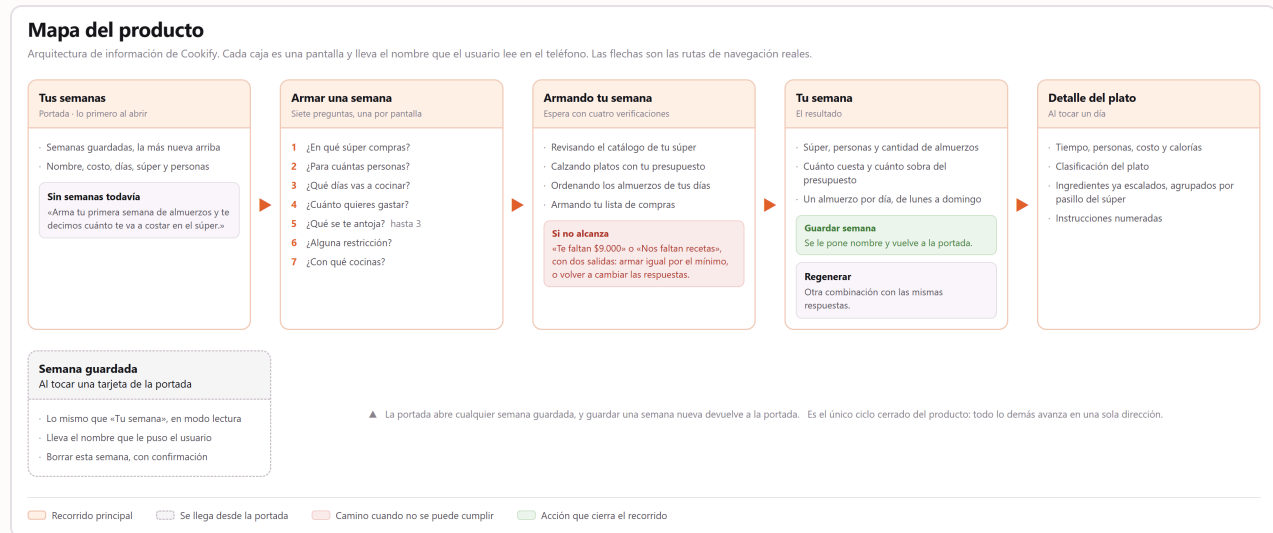


Figura 1. Mapa del producto. Cada caja es una pantalla y lleva el nombre que el usuario lee en el teléfono; las flechas son las rutas de navegación reales.

La regla de nombres

Cada sección se nombra con el vocabulario del usuario y no con el del sistema. La portada se llama «**Tus semanas**» y no «Listado de planes»; la espera se llama «**Armando tu semana**» y no «Procesando». Los nombres técnicos —`SemanaRepository`, `GenerarPlanSemanaUseCase`— existen, pero viven en el capítulo 4 y nunca en la pantalla.

La jerarquía

El producto tiene **un solo nivel de navegación principal**. No hay menú lateral ni pestañas, porque la aplicación hace una cosa y esconder esa cosa detrás de un menú sería trabajo adicional para el usuario sin beneficio.

Desde la portada salen exactamente dos caminos: armar una semana nueva, o abrir una guardada. Todo lo demás son pasos dentro de esos dos caminos.

El único ciclo cerrado

La navegación avanza en una sola dirección salvo en un punto: **guardar una semana devuelve a la portada**, y desde la portada se puede abrir esa misma semana. Ese ciclo es deliberado: es el que convierte un uso puntual en un hábito, porque la siguiente vez que la persona abra la aplicación lo primero que verá es lo que armó la semana pasada.

UNA DECISIÓN DE AGRUPACIÓN

Las siete preguntas podrían haber sido una sola pantalla con siete campos. Se separaron porque cada respuesta cambia lo que la siguiente significa —el presupuesto se juzga distinto según cuántas personas y cuántos días— y porque una pantalla con siete campos vacíos se abandona.

CAPÍTULO 4

Arquitectura técnica

Cliente, servidor y capas, con los nombres reales de los componentes del repositorio.

4.1 Cliente, servidor y capas

Cookify es una aplicación **cliente–servidor en capas**. El cliente es una aplicación Android nativa que hoy funciona completa y sin red. El servidor es un servicio de precios que se construye en el Hito 2 y cuya necesidad está justificada con mediciones propias en la sección 4.3.

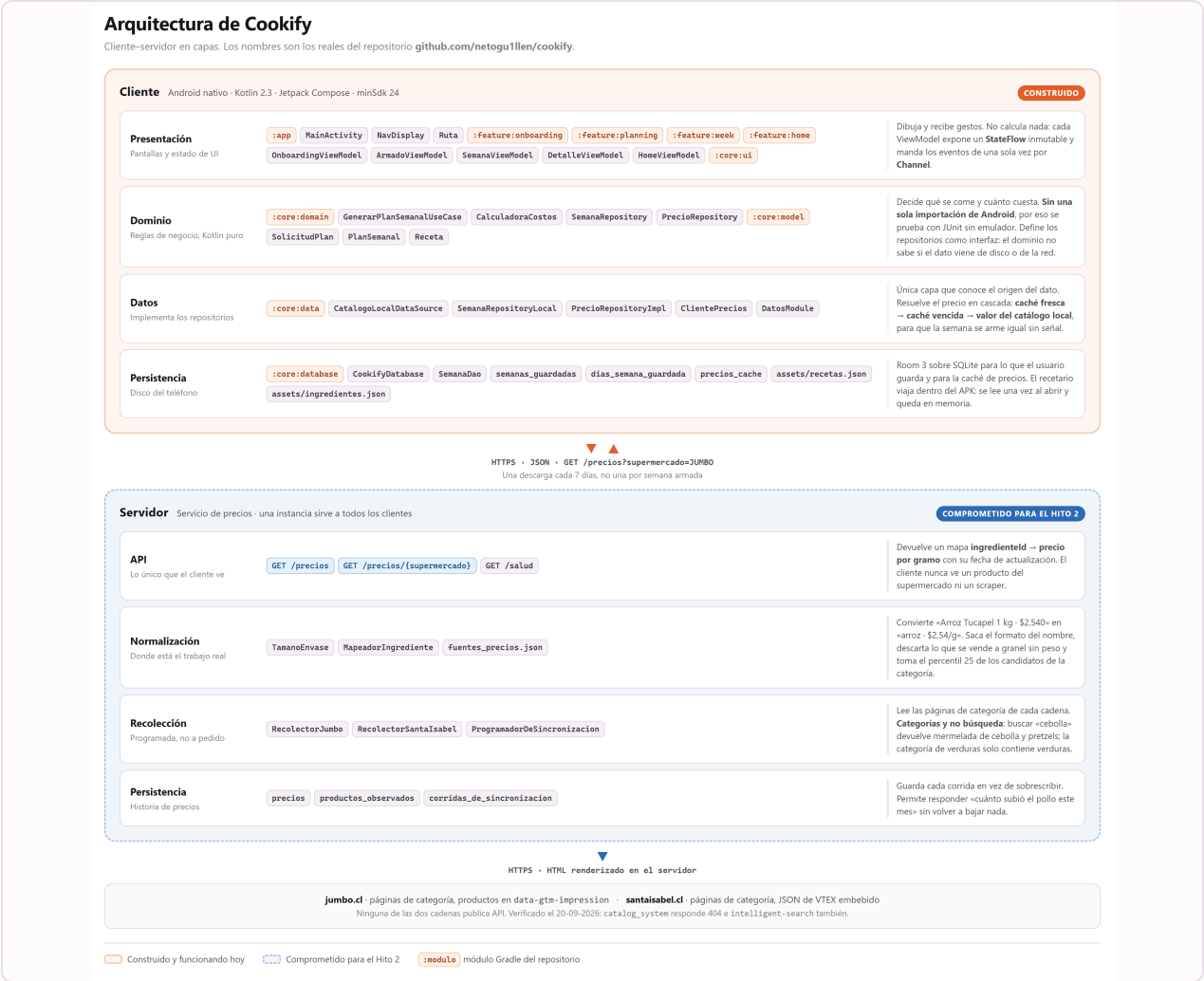


Figura 2. Arquitectura cliente–servidor en capas. En ámbar sólido, lo que ya existe; en azul punteado, lo comprometido para el Hito 2. Los nombres son los de los módulos y clases del repositorio.

La regla que ordena el diseño es simple: **cada capa solo conoce la que tiene debajo, y el dominio no conoce a ninguna**. El módulo `:core:domain` no tiene una sola importación de Android, de modo que el motor de planificación se prueba con JUnit en la máquina virtual de Java, sin emulador.

Responsabilidades de cada capa

CAPA	MÓDULOS	DE QUÉ RESPONDE	DE QUÉ NO
Presentación	<code>:app</code> , <code>:feature:*</code> , <code>:core:ui</code>	Dibujar y recibir gestos	No calcula precios ni elige recetas
Dominio	<code>:core:domain</code> , <code>:core:model</code>	Qué se come y cuánto cuesta	No sabe si el dato viene de disco o de red
Datos	<code>:core:data</code>	Resolver de dónde sale cada dato	No decide reglas de negocio
Persistencia	<code>:core:database</code> , <code>assets/</code>	Guardar y leer	No transforma nada

El dominio define `SemanaRepository` y `PrecioRepository` como **interfaces**, y la capa de datos las implementa. Esa inversión de dependencias es lo que permitirá que el Hito 2 agregue el servidor **sin tocar el motor de planificación**: cambia quién responde el precio, no quién lo usa.

4.2 Qué hace cada capa al armar una semana

El recorrido completo del producto, desde el toque final del onboarding hasta la portada actualizada.

#	CAPA	QUÉ OCURRE
1	Presentación	<code>OnboardingViewModel</code> guarda cada respuesta en un <code>SavedStateHandle</code> , que sobrevive a que Android mate el proceso, y valida en cada paso si se puede avanzar. Al terminar emite <code>EventoOnboarding.Listo(SolicitudPlan)</code> .
2	Presentación	<code>MainActivity</code> navega a <code>Ruta.Armado</code> . La ruta lleva la solicitud y una semilla, nunca el plan ya construido : así la navegación se serializa barata y el plan se reconstruye igual en cualquier momento.
3	Dominio	<code>GenerarPlanSemanaUseCase</code> filtra el recetario por restricción y artefactos, puntúa cada receta según los antojos, selecciona sin repetir plato ni abusar de una proteína, y ajusta a presupuesto. Es determinista: misma semilla, mismo plan.
4	Datos	<code>CatalogoLocalDataSource</code> entrega las 69 recetas y los 98 ingredientes. Se leen una sola vez al abrir la aplicación y quedan en memoria: son de solo lectura, así que no pasan por la base de datos.
5	Datos → Red	<code>PrecioRepositoryImpl</code> resuelve el precio en cascada: caché fresca de menos de 7 días, luego caché vencida, y por último el valor del catálogo local. Si hay red y la caché venció, se pide al servidor en segundo plano. (<i>Hito 2</i>)

Qué hace cada capa al armar una semana · continuación

#	CAPA	QUÉ OCURRE
6	Dominio	<code>CalculadoraCostos</code> escala cada ingrediente por el número de personas y redondea hacia arriba lo que se vende por unidad : media palta no existe en el supermercado, y el presupuesto debe cobrar la palta entera que se va a pagar.
7	Presentación	El motor termina en milisegundos, así que la espera escenifica cuatro verificaciones que nombran los datos reales del usuario. Si el presupuesto no alcanza, la secuencia se detiene en la etapa que falló y ofrece el mínimo viable calculado.
8	Persistencia	Al guardar, <code>SemanaRepositoryLocal</code> escribe por <code>SemanaDao</code> en dos tablas con borrado en cascada. Se guarda el plato de cada día, no la semilla : si mañana cambia el recetario, la semana guardada no muta.
9	Presentación	<code>HomeViewModel</code> observa un <code>Flow</code> del repositorio, de modo que la portada se actualiza sola sin que nadie le avise.

Dónde se nota el corte en capas

Un corte en capas se justifica cuando permite cambiar una pieza sin tocar las otras. En este proyecto eso ya ocurrió tres veces:

- **El motor se prueba sin emulador.** Como el dominio no importa nada de Android, las pruebas del planificador corren en la máquina virtual de Java en milisegundos, no en un emulador que tarda minutos en arrancar.
- **El precio va a cambiar de origen sin que el motor se entere.** Hoy sale de un archivo en el teléfono; en el Hito 2 saldrá de un servidor. El motor pide un precio a una interfaz y no sabe ni le importa quién la implementa.
- **El recetario se valida sin abrir la aplicación.** Una prueba verifica que cada ingrediente citado por una receta exista realmente en el catálogo de precios. Esa prueba ya encontró un error que habría llegado al usuario.

4.3 Por qué el precio necesita un servidor

Esta es la decisión arquitectónica central del proyecto, y no se tomó por conveniencia académica sino a partir de una medición propia.

La medición

El modelo inicial estimaba el precio de cada cadena multiplicando un precio base por una constante: Jumbo 1,15, Santa Isabel 1,08, y así. Para verificar si esa constante existe, se compararon los **17 productos idénticos** presentes el mismo día en el catálogo en línea de Jumbo y de Santa Isabel.

PRODUCTO	JUMBO	SANTA ISABEL	DIFERENCIA
Arroz Taj Mahal basmati 1 kg	\$5.890	\$5.890	0,0 %
Arroz Miraflores grano largo y ancho 1 kg	\$2.650	\$2.690	+1,5 %
Arroz Banquete grado 2 1 kg	\$2.300	\$2.090	−9,1 %
Arroz Miraflores basmati 400 g	\$2.430	\$2.990	+23,0 %
Arroz Tucapel gran selección 1 kg	\$1.790	\$2.540	+41,9 %
Arroz Tucapel Blue 1 kg	\$1.150	\$1.810	+57,4 %

La diferencia va de **−9,1 % a +57,4 %**, y **depende del producto y no de la cadena**. No existe un multiplicador por supermercado. Si el precio no se puede derivar de una constante, hay que **medirlo**.

Por qué medirlo desde el teléfono no sirve

Se midió también el costo de que cada teléfono hiciera ese trabajo por su cuenta:

OPERACIÓN	PESO COMPRIMIDO	TIEMPO
Una categoría de Jumbo	302 KB	2,8 s
Una búsqueda de Santa Isabel	41 KB	1,1 s
Un refresco completo de precios	6–9 MB	10–20 s

Tres razones para que eso viva en un servidor: **el costo por cliente** (6–9 MB por usuario contra una descarga que sirve a todos), **la fragilidad** (cuando una cadena cambia su sitio se cae un servicio que se arregla en una tarde, no una aplicación instalada que necesita pasar por la tienda) y **la cortesía de red** (un servicio consulta cada varias horas; mil teléfonos consultando es otra cosa).

Lo que el servidor hace y el cliente no ve

La API devuelve un mapa de ingredienteId a precio por gramo, con su fecha de actualización. El cliente **nunca ve un producto de supermercado**: no sabe que existe «Arroz Tucapel 1 kg», solo que el arroz cuesta \$2,54 por gramo en Jumbo.

Entre una cosa y la otra hay dos problemas resueltos que justifican la capa de normalización:

- **Sacar el formato del nombre.** «Arroz Tucapel Pregraneado 1 kg» vale \$2.540, pero el recetario pide gramos. Hay que leer el tamaño del envase desde el nombre del producto y descartar lo que se vende a granel sin peso declarado.
- **Leer categorías y no búsquedas.** Buscar «cebolla» en el sitio de un supermercado devuelve mermelada de cebolla, pretzels con cebolla y condimento en polvo. La categoría de verduras, en cambio, solo contiene verduras. Es un filtro estructural en lugar de una lista negra de palabras que nunca termina.

POR QUÉ ESTO IMPORTA MÁS DE LO QUE PARECE

Un error de mapeo aquí *no falla*: cobra mal en silencio. En las pruebas, una búsqueda mal filtrada valoró la leche a \$8,74 por mililitro cuando la real cuesta \$1,06. Una receta con medio litro de leche habría pasado de \$650 a \$4.370 sin que nada se cayera. Por eso la normalización es una capa con su propia responsabilidad y sus propias pruebas, y no un detalle del recolector.

Decisión de renderizado

La Unidad 1 pide decidir entre aplicación de página única y multipágina. El cliente de Cookify es **nativo, no web**, así que la decisión equivalente es entre una sola *Activity* con navegación declarativa y varias *Activities* independientes.

Se eligió **una sola Activity**, que es el análogo directo de la aplicación de página única: hay un solo contenedor y el contenido se reemplaza dentro de él. La razón concreta es el onboarding: sus siete pasos comparten un mismo estado, y con *Activities* separadas ese estado tendría que serializarse y reconstruirse en cada paso. El costo asumido se detalla en la sección 5.2.

CAPÍTULO 5

Decisiones de tecnología

Cuatro decisiones, cada una con su alternativa comparada, sus criterios y la desventaja que se asume por escrito.

5.1 Plataforma del cliente

Android nativo con Kotlin y Jetpack Compose, frente a React Native.

CRITERIO	ANDROID NATIVO	REACT NATIVE
Costo de la primera versión funcionando Un solo desarrollador, un semestre	Una plataforma, un lenguaje, una cadena de compilación	Dos plataformas de destino y una capa intermedia que también hay que aprender
Trabajo sin red y almacenamiento local Es un requisito del producto, no un extra	Room y DataStore son de primera clase	Depende de bibliotecas de terceros sobre SQLite
Riesgo de la capa intermedia Qué se rompe cuando Android cambia	Ninguna capa entre el código y el sistema	Cada versión de Android puede romper el puente antes de que lo arreglen

DECISIÓN FIRMADA · DESVENTAJA ASUMIDA

Se elige **Android nativo**. La desventaja que se asume es concreta y cara: **no habrá versión de iOS ni versión web, y no las habrá sin reescribir el cliente completo**. Se acepta porque el producto tiene que funcionar sin red y guardar datos localmente, y porque con un solo desarrollador la profundidad en una plataforma vale más que la cobertura superficial en dos.

5.2 Navegación y renderizado

Navigation3, frente a Navigation Compose 2.

CRITERIO	NAVIGATION3	NAVIGATION COMPOSE 2
Cómo se pasan los datos entre pantallas	Objetos tipados en la pila de navegación	Cadenas de texto en una ruta, que hay que codificar y volver a interpretar
Control de la pila	La pila es una lista que el código manipula directamente	Se opera a través de un controlador con su propia gramática
Madurez	Versión 1.1.4, API joven	Años de uso y documentación

DECISIÓN FIRMADA · DESVENTAJA ASUMIDA

Se elige **Navigation3** por el paso de datos tipado: las rutas de Cookify llevan objetos completos —la solicitud del plan, la semilla— y codificarlos como texto habría sido una fuente permanente de errores.

La desventaja no es teórica: **ya se pagó**. Navigation3 instala por defecto un solo decorador de entradas, y sin el de *ViewModelStore* todas las pantallas comparten la misma instancia de cada modelo de vista. El síntoma en la aplicación fue que al abrir una segunda receta se mostraba la primera. Detectarlo exigió leer el código fuente de la biblioteca, porque no está documentado. Con una biblioteca madura ese problema no habría existido.

5.3 Persistencia y servidor

Persistencia local: Room 3 frente a archivos JSON

El producto guarda semanas con sus días, y cada día apunta a una receta. Es una relación de uno a muchos que hay que leer ordenada y borrar en cascada.

CRITERIO	ROOM 3	ARCHIVOS JSON
Relaciones y borrado en cascada	Nativo, verificado al compilar	A mano, y un descuido deja días huérfanos
Reactividad	Devuelve flujos: la portada se actualiza sola	Hay que releer el archivo y avisar manualmente
Ceremonia	Procesador de anotaciones, esquemas versionados, migraciones	Leer y escribir un archivo

DECISIÓN FIRMADA · DESVENTAJA ASUMIDA

Se elige **Room 3**. La desventaja asumida es la ceremonia: para un modelo de dos tablas hay que mantener un procesador de anotaciones, esquemas versionados y migraciones. Se acepta porque la portada tiene que actualizarse sola —y eso lo dan los flujos de Room— y porque el borrado en cascada verificado al compilar evita una clase entera de errores.

Servidor del Hito 2: Kotlin con Ktor frente a Python con FastAPI

CRITERIO	KOTLIN · KTOR	PYTHON · FASTAPI
Compartir el modelo con la aplicación	<code>: core : model</code> es Kotlin puro y se comparte tal cual	Hay que duplicar los tipos, y con el tiempo se desalinean
Herramientas para leer HTML	Jsoup, correcto pero más verboso	Mejor ecosistema, y ya hay prototipos funcionando
Un solo lenguaje en el proyecto	Todo el repositorio en Kotlin	Dos lenguajes, dos cadenas de herramientas

DECISIÓN FIRMADA · DESVENTAJA ASUMIDA

Se elige **Kotlin con Ktor**, para que **Ingrediente**, **Pasillo** y **Unidad** sean literalmente el mismo código en el servidor y en la aplicación, sin posibilidad de que se desalineen. La desventaja asumida es que **se descartan los prototipos de recolección ya escritos en Python** y que leer HTML en Kotlin es más incómodo. Se acepta porque un desajuste silencioso entre el modelo del servidor y el de la aplicación se manifestaría como precios equivocados, que es exactamente el error que este proyecto no se puede permitir.

Resumen de la pila

CAPA	HERRAMIENTA	POR QUÉ
Lenguaje	Kotlin 2.3	Nulabilidad en el tipo y corrutinas en el lenguaje
Interfaz	Jetpack Compose	La interfaz es una función del estado, no un árbol mutable
Navegación	Navigation3 1.1.4	Rutas tipadas, pila manipulable
Inyección	Hilt	Grafo verificado al compilar
Base de datos	Room 3	Relaciones, flujos y migraciones verificadas
Serialización	kotlinx.serialization	Sin reflexión en tiempo de ejecución
Red (<i>Hito 2</i>)	Ktor Client	Mismo lenguaje en los dos extremos
Servidor (<i>Hito 2</i>)	Ktor Server	Comparte el módulo de modelo con la aplicación
Calidad	detekt, JUnit, Turbine	Análisis estático y pruebas en cada compilación

Lo que la arquitectura hace posible

La coherencia entre el problema, la experiencia prometida y la arquitectura elegida se puede leer en tres afirmaciones concretas:

- El producto promete funcionar en el supermercado, donde la señal es mala. Por eso el recetario viaja dentro de la aplicación y el precio se resuelve en cascada hasta un valor local: **la semana se arma igual sin red**.
- El producto promete un número confiable. Por eso el precio se mide en un servidor en lugar de estimarse con una constante, y por eso la normalización es una capa con pruebas propias.
- El producto promete no hacer perder tiempo. Por eso el motor es determinista y corre en milisegundos, y la espera que el usuario ve es una decisión de diseño y no una limitación técnica.

CAPÍTULO 6

Lo que viene

Lo que se compromete para los hitos 2 y 3, con el criterio de que lo prometido aquí es lo que se entrega después.

Hito 2 · El servicio de precios

Construir el servidor descrito en la sección 4.3:

- API en Ktor que sirva el mapa de precios por cadena, compartiendo `:core:model` con la aplicación.
- Recolectores para Jumbo y Santa Isabel que lean páginas de categoría, con el árbol de categorías de alimentos ya identificado.
- Capa de normalización que convierta «producto de supermercado» en «precio por gramo», con sus pruebas.
- Mapeo de los 98 ingredientes a su categoría y sus términos de búsqueda. Es el trabajo más delicado del hito, porque un mapeo equivocado no falla: cobra mal.
- Historia de precios: cada corrida se guarda en vez de sobrescribir.
- En la aplicación: caché en Room con vencimiento, sincronización en segundo plano y marca visible de cuándo se actualizaron los precios.

Hito 3 · El producto que funciona

- **Lista de compras consolidada de la semana**, agrupada por pasillo, que hoy la aplicación anuncia durante el armado pero todavía no entrega.
- Declarar en la interfaz cuándo un precio es medido y cuándo es estimado, con la fecha de la última actualización.
- Recordar las respuestas del recorrido entre semanas: quien compra siempre en el mismo supermercado no debería contestar siete preguntas cada vez.
- Repetir o duplicar una semana guardada.
- Accesibilidad verificada con lector de pantalla y R8 activado para la versión de publicación.

UNA DEUDA QUE SE DECLARA

Una revisión de usabilidad detectó que la pantalla de armado anuncia «Armando tu lista de compras» y esa lista todavía no existe en ninguna pantalla. Se deja consignado aquí en lugar de esconderlo: es la primera deuda que el Hito 3 salda.

CONCLUSIÓN

Una promesa que ya se puede verificar

Cookify resuelve un problema que tiene tamaño medido: **\$307.947 mensuales** por hogar en alimentos, la primera categoría de gasto del presupuesto familiar chileno y la que más creció. Y resuelve una parte del problema que nadie está atendiendo: *no qué cocinar*, sino *si alcanza*.

La evidencia del SERNAC —**237,8 % de diferencia** por la misma canasta de seis productos según dónde se compre— convierte al supermercado en una variable de primer orden, y esa es exactamente la primera pregunta que el producto hace.

Lo que distingue esta propuesta

Al cierre del Hito 1 hay un recetario de 69 platos chilenos, un motor de planificación determinista con pruebas, persistencia local y un camino resuelto para cuando el presupuesto no alcanza.

Y la decisión arquitectónica central —que el precio necesita un servidor— **no se tomó porque un servidor se vea bien en un diagrama**. Se tomó después de medir que la diferencia de precio entre dos cadenas va de $-9,1\%$ a $+57,4\%$ según el producto, lo que demuestra que no existe la constante que el modelo anterior suponía. Esa medición es propia, es reproducible y está en el repositorio.

La arquitectura no se dibujó para ilustrar el producto. Se eligió para resolver un problema que primero se midió.

Lo que queda por delante

El Hito 2 construye el servicio que hace verdadera la palabra «real» en la promesa. El Hito 3 salda las deudas que esta propuesta declara en lugar de esconder, empezando por la lista de compras que la aplicación anuncia y todavía no entrega.

El compromiso de este documento es que lo prometido aquí es lo que se entrega en diciembre. Lo que ya está construido está señalado como construido, y lo que falta está señalado como falta, en cada sección donde aparece.

BIBLIOGRAFÍA

Referencias

Norma APA, séptima edición. Todas las fuentes se verificaron abriendo el documento original el 20 de septiembre de 2026.

Biblioteca del Congreso Nacional de Chile. (2021). *Proyectos de ley que regulen el desperdicio alimentario en Chile: antecedentes y actualidad* (Minuta N.º 32-21). Departamento de Estudios, Extensión y Publicaciones. https://obtienearchivo.bcn.cl/obtienearchivo?id=repositorio/10221/32197/1/Minuta_32_21_Desperdicio_Alimentos_Chile.pdf

Google. (2026). *Jetpack Compose: documentación para desarrolladores de Android*. <https://developer.android.com/develop/ui/compose/documentation>

Google. (2026). *Navigation 3: navegación declarativa para Jetpack Compose*. <https://developer.android.com/guide/navigation/navigation-3>

Google. (2026). *Room: guía de persistencia local en Android*. <https://developer.android.com/training/data-storage/room>

Instituto Nacional de Estadísticas. (2023). *IX Encuesta de Presupuestos Familiares: síntesis de resultados (octubre 2021 – septiembre 2022)*. INE. <https://www.ine.gob.cl/epf>

JetBrains. (2026). *Ktor: framework asíncrono para aplicaciones conectadas en Kotlin*. <https://ktor.io/docs/welcome.html>

Servicio Nacional del Consumidor. (2026, 7 de septiembre). *Cotizador de Fiestas Patrias: casi \$55 mil de diferencia en la «canasta dieciochera» detectó el SERNAC*. SERNAC. <https://www.sernac.cl/portal/604/w3-article-89167.html>

Fuente primaria propia

La comparación de precios entre Jumbo y Santa Isabel presentada en la sección 4.3 es una medición realizada por el autor el 20 de septiembre de 2026 sobre los catálogos en línea públicos de ambas cadenas. El procedimiento y los datos crudos están documentados en el repositorio del proyecto, en `docs/PRECIOS_REALES.md` y `docs/evidencia.md`.

ANEXOS

Anexos

Anexo A · Dónde está cada cosa

RECURSO	UBICACIÓN
Repositorio	github.com/netogu11en/cookify
Prototipo navegable	netogu11en.github.io/cookify/fuentes/prototipo.html
Este documento	docs/propuesta-cookify.pdf
Láminas de la presentación	docs/laminas-hito1.pdf
Mapa del producto	docs/mapa-del-producto.png
Diagrama de arquitectura	docs/arquitectura.png
Arquitectura en detalle	docs/arquitectura.md
Evidencia del problema	docs/evidencia.md
Diseño del servicio de precios	docs/PRECIOS_REALES.md
Capturas de la aplicación	docs/capturas/

Anexo B · Cómo compilar y ejecutar

Requiere JDK 21 y el SDK de Android con la plataforma 37.

COMANDO	QUÉ HACE
<code>./gradlew :app:assembleDebug</code>	Compila el APK de depuración
<code>./gradlew test</code>	Pruebas del motor, los modelos de vista y el catálogo
<code>./gradlew detekt</code>	Análisis estático, incluidas las reglas de Compose

Anexo C · Módulos del repositorio

MÓDULO	CONTENIDO
:app	Navegación y arranque de la aplicación
:feature:onboarding	Las siete preguntas
:feature:planning	El armado y sus verificaciones
:feature:week	La semana y el detalle de cada plato
:feature:home	Las semanas guardadas
:core:model	Modelos del dominio, Kotlin puro
:core:domain	Motor de planificación y cálculo de costos
:core:data	Catálogo, repositorios y precios
:core:database	Room 3: entidades, DAO y base de datos
:core:ui	Tema oscuro y componentes compartidos
:core:common	Utilidades y formato de moneda

Anexo D · Declaración de uso de inteligencia artificial

Conforme a la regla 7 del proyecto. Se utilizó Claude (Anthropic) como asistente de programación y de redacción. Las decisiones de arquitectura, la selección de tecnologías y la verificación de cada afirmación son del autor y están documentadas en el repositorio. Las cifras del capítulo 1 se verificaron abriendo la fuente original en cada caso; las notas de prensa sirvieron para localizar los estudios, no para citarlos. La medición de dispersión de precios de la sección 4.3 es propia y reproducible. El detalle completo de la declaración está en el archivo `README.md` del repositorio.